

# btt006\_\_week5\_\_assignment-solution

May 8, 2026

## 1 Assignment 5: ML Life Cycle: Evaluation and Deployment

```
[1]: import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
```

In this assignment, you will continue practicing the fifth step of the machine learning life cycle. You will perform model selection for logistic regression to solve a classification problem. You will complete the following tasks:

1. Build your DataFrame and define your ML problem:
  - Load the Airbnb "listings" data set
  - Define the label - what are you predicting?
  - Identify the features
2. Create labeled examples from the data set
3. Split the data into training and test data sets
4. Train, test and evaluate a logistic regression (LR) model using the scikit-learn default value for hyperparameter  $C$
5. Perform a grid search to identify the optimal value of  $C$  for a logistic regression model
6. Train, test and evaluate a logistic regression model using the optimal value of  $C$
7. Plot a precision-recall curve for both models
8. Plot the ROC and compute the AUC for both models
9. Perform feature selection
10. Make your model persistent for future use

**Note:** Some of the code cells in this notebook may take a while to run.

### 1.1 Part 1. Build Your DataFrame and Define Your ML Problem

**Load a Data Set and Save it as a Pandas DataFrame** We will work with the data set `airbnbData_train`. This data set already has all the necessary preprocessing steps implemented,

including one-hot encoding of the categorical variables, scaling of all numerical variable values, and imputing missing values. It is ready for modeling.

```
[2]: filename = os.path.join(os.getcwd(), "data_LR", "airbnbData_train.csv")
     df = pd.read_csv(filename, header=0)
```

**Define the Label** Your goal is to train a machine learning model that predicts whether an Airbnb host is a 'super host'. This is an example of supervised learning and is a binary classification problem. In our dataset, our label will be the `host_is_superhost` column and the label will either contain the value `True` or `False`.

**Identify Features** Our features will be all of the remaining columns in the dataset.

## 1.2 Part 2. Create Labeled Examples from the Data Set

Task: In the code cell below, create labeled examples from DataFrame `df`. Assign the labels to variable `y` and the features to variable `X`.

```
[3]: # YOUR CODE HERE

#SOLUTION:

y = df['host_is_superhost']
X = df.drop(columns = 'host_is_superhost', axis=1)
```

## 1.3 Part 3. Create Training and Test Data Sets

Task: In the code cell below, create training and test sets out of the labeled examples. Create a test set that is 10 percent of the size of the data set. Save the results to variables `X_train`, `X_test`, `y_train`, `y_test`.

```
[4]: # YOUR CODE HERE

#SOLUTION:

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.10,
    random_state=1234)
```

## 1.4 Part 4. Train, Test and Evaluate a Logistic Regression Model With Default Hyperparameter Values

You will fit a logistic regression model to the training data using scikit-learn's default value for hyperparameter `C`. You will then make predictions on the test data and evaluate the model's per-

formance. The goal is to later find a value for hyperparameter  $C$  that can improve this performance of the model on the test data.

Task: In the code cell below:

1. Using the scikit-learn `LogisticRegression` class, create a logistic regression model object with the following arguments: `max_iter=1000`. You will use the scikit-learn default value for hyperparameter  $C$ , which is 1.0. Assign the model object to the variable `model_default`.
2. Fit the model to the training data.

```
[5]: # YOUR CODE HERE

#Solution:
model_default = LogisticRegression(max_iter=1000)
model_default.fit(X_train, y_train)
```

```
[5]: LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
                        intercept_scaling=1, l1_ratio=None, max_iter=1000,
                        multi_class='auto', n_jobs=None, penalty='l2',
                        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                        warm_start=False)
```

Task: Test your model on the test set (`X_test`).

1. Use the `predict_proba()` method to use the fitted model to predict class probabilities for the test set. Note that the `predict_proba()` method returns two columns, one column per class label. The first column contains the probability that an unlabeled example belongs to class `False` (`host_is_superuser` is "False") and the second column contains the probability that an unlabeled example belongs to class `True` (`host_is_superuser` is "True"). Save the values of the *second* column to a list called `proba_predictions_default`.
2. Use the `predict()` method to use the fitted model `model_default` to predict the class labels for the test set. Store the outcome in the variable `class_label_predictions_default`. Note that the `predict()` method returns the class label (True or False) per unlabeled example.

```
[6]: # 1. Make predictions on the test data using the predict_proba() method
# YOUR CODE HERE

### Solution:
proba_predictions = model_default.predict_proba(X_test)
proba_predictions_default = []
for i in proba_predictions:
    proba_predictions_default.append(i[1])

# 2. Make predictions on the test data using the predict() method
# YOUR CODE HERE

class_label_predictions_default = model_default.predict(X_test)
```

Task: Evaluate the accuracy of the model using a confusion matrix. In the cell below, create

a confusion matrix out of `y_test` and `class_label_predictions_default`. Use the Confusion Matrix Demo as a reference.

```
[7]: # YOUR CODE HERE

# solution (solutions may vary)
c_m = confusion_matrix(y_test, class_label_predictions_default, labels=[True,
↪False])

pd.DataFrame(
    c_m,
    columns=['Predicted: Terrible Host', 'Predicted: Great Host'],
    index=['Actual: Terrible Host', 'Actual: Great Host']
)
```

```
[7]:
```

|                       | Predicted: Terrible Host | Predicted: Great Host |
|-----------------------|--------------------------|-----------------------|
| Actual: Terrible Host | 265                      | 450                   |
| Actual: Great Host    | 91                       | 1997                  |

## 1.5 Part 5. Perform Logistic Regression Model Selection Using `GridSearchSV()`

Our goal is to find the optimal choice of hyperparameter  $C$ . Our goal is to find the optimal choice of hyperparameter  $C$ . We will then fit a logistic regression model to the training data using this value of  $C$ .

### 1.5.1 Set Up a Parameter Grid

Task: Create a dictionary called `param_grid` that contains 10 possible hyperparameter values for  $C$ . The dictionary should contain the following key/value pair:

- a key called  $C$
- a value which is a list consisting of 10 values for the hyperparameter  $C$ . A smaller value for “ $C$ ” (e.g.  $C=0.01$ ) leads to stronger regularization and a simpler model, while a larger value (e.g.  $C=1.0$ ) leads to weaker regularization and a more complex model. Use the following values for  $C$ : `cs=[10**i for i in range(-5,5)]`

```
[8]: # YOUR CODE HERE

# SOLUTION
cs=[10**i for i in range(-5,5)]
param_grid = dict(C = list(cs))

param_grid
```

```
[8]: {'C': [1e-05, 0.0001, 0.001, 0.01, 0.1, 1, 10, 100, 1000, 10000]}
```

### 1.5.2 Perform Grid Search Cross-Validation

Task: Use `GridSearchCV` to search over the different values of hyperparameter  $C$  to find the one that results in the best cross-validation (CV) score.

Complete the code in the cell below. Note: This will take a few minutes to run.

```
[9]: print('Running Grid Search...')

# 1. Create a LogisticRegression model object with the argument max_iter=1000.
#     Save the model object to the variable 'model'
# YOUR CODE HERE

#Solution:
model = LogisticRegression(max_iter=1000)

# 2. Run a grid search with 5-fold cross-validation and assign the output to
#     the
# object 'grid'.
#     * Pass the model and the parameter grid to GridSearchCV()
#     * Set the number of folds to 5

#Solution:
grid = GridSearchCV(model, param_grid, cv=5)

# 3. Fit the model (use the 'grid' variable) on the training data and assign
#     the fitted model to the
#     variable 'grid_search'

#Solution:
grid_search = grid.fit(X_train, y_train)

print('Done')
```

Running Grid Search...

Done

Task: Retrieve the value of the hyperparameter  $C$  for which the best score was attained. Save the result to the variable `best_c`.

```
[10]: # YOUR CODE HERE

### Solution:
best_C = grid_search.best_params_['C']
best_C
```

```
[10]: 1000
```

## 1.6 Part 6. Train, Test and Evaluate the Optimal Logistic Regression Model

Now that we have the optimal value for hyperparameter  $C$ , let's train a logistic regression model using that value, test the model on our test data, and evaluate the model's performance.

Task: Initialize a `LogisticRegression` model object with the best value of hyperparameter  $C$  model and fit the model to the training data. The model object should be named `model_best`.

Note: Supply `max_iter=1000` as an argument when creating the model object.

```
[11]: # YOUR CODE HERE

# solution:
model_best = LogisticRegression(C = best_C, max_iter=1000)
model_best.fit(X_train, y_train)
```

```
[11]: LogisticRegression(C=1000, class_weight=None, dual=False, fit_intercept=True,
                        intercept_scaling=1, l1_ratio=None, max_iter=1000,
                        multi_class='auto', n_jobs=None, penalty='l2',
                        random_state=None, solver='lbfgs', tol=0.0001, verbose=0,
                        warm_start=False)
```

Task: Test your model on the test set (`X_test`).

1. Use the `predict_proba()` method to use the fitted model `model_best` to predict class probabilities for the test set. Save the values of the *second* column to a list called `proba_predictions_best`.
2. Use the `predict()` method to use the fitted model `model_best` to predict the class labels for the test set. Store the outcome in the variable `class_label_predictions_best`.

```
[12]: # 1. Make predictions on the test data using the predict_proba() method
      # YOUR CODE HERE

#Solution (one potential solution):
proba_predictions = model_best.predict_proba(X_test)
proba_predictions_best = []
for i in proba_predictions:
    proba_predictions_best.append(i[1])

# 2. Make predictions on the test data using the predict() method
# YOUR CODE HERE

#Solution:
class_label_predictions_best = model_best.predict(X_test)
```

Task: Evaluate the accuracy of the model using a confusion matrix. In the cell below, create a confusion matrix out of `y_test` and `class_label_predictions_best`.

```
[13]: # YOUR CODE HERE

#Solution:

c_m = confusion_matrix(y_test, class_label_predictions_best, labels=[True,
→False])

pd.DataFrame(
c_m,
columns=['Predicted: Terrible Host', 'Predicted: Great Host'],
index=['Actual: Terrible Host', 'Actual: Great Host']
)
```

```
[13]:
```

|                       | Predicted: Terrible Host | Predicted: Great Host |
|-----------------------|--------------------------|-----------------------|
| Actual: Terrible Host | 271                      | 444                   |
| Actual: Great Host    | 88                       | 2000                  |

## 1.7 Part 7. Plot Precision-Recall Curves for Both Models

Recall that scikit-learn defaults to a 0.5 classification threshold. Sometimes we may want a different threshold. We can use the precision-recall curve to show the trade-off between precision and recall for different classification thresholds. Scikit-learn's `precision_recall_curve()` function computes precision-recall pairs for different probability thresholds. For more information, consult the [Scikit-learn documentation](#).

Let's first import the function.

```
[14]: from sklearn.metrics import precision_recall_curve
```

Task: In the code cell below, use `precision_recall_curve()` to compute precision-recall pairs for both models.

For `model_default`: \* call `precision_recall_curve()` with `y_test` and `proba_predictions_default` \* save the output to the variables `precision_default`, `recall_default` and `thresholds_default`, respectively

For `model_best`: \* call `precision_recall_curve()` with `y_test` and `proba_predictions_best` \* save the output to the variables `precision_best`, `recall_best` and `thresholds_best`, respectively

```
[15]: #precision_default, recall_default, thresholds_default = # YOUR CODE HERE
#precision_best, recall_best, thresholds_best = # YOUR CODE HERE

# solution:
```

```
precision_default, recall_default, thresholds_default=precision_recall_curve(y_test, proba_predictions_default)
precision_best, recall_best, thresholds_best = precision_recall_curve(y_test, proba_predictions_best)
```

In the code cell below, create two **seaborn** lineplots to visualize the precision-recall curve for both models. "Recall" will be on the *x*-axis and "Precision" will be on the *y*-axis.

The plot for "default" should be green. The plot for the "best" should be red.

```
[16]: # YOUR CODE HERE

# begin solution:

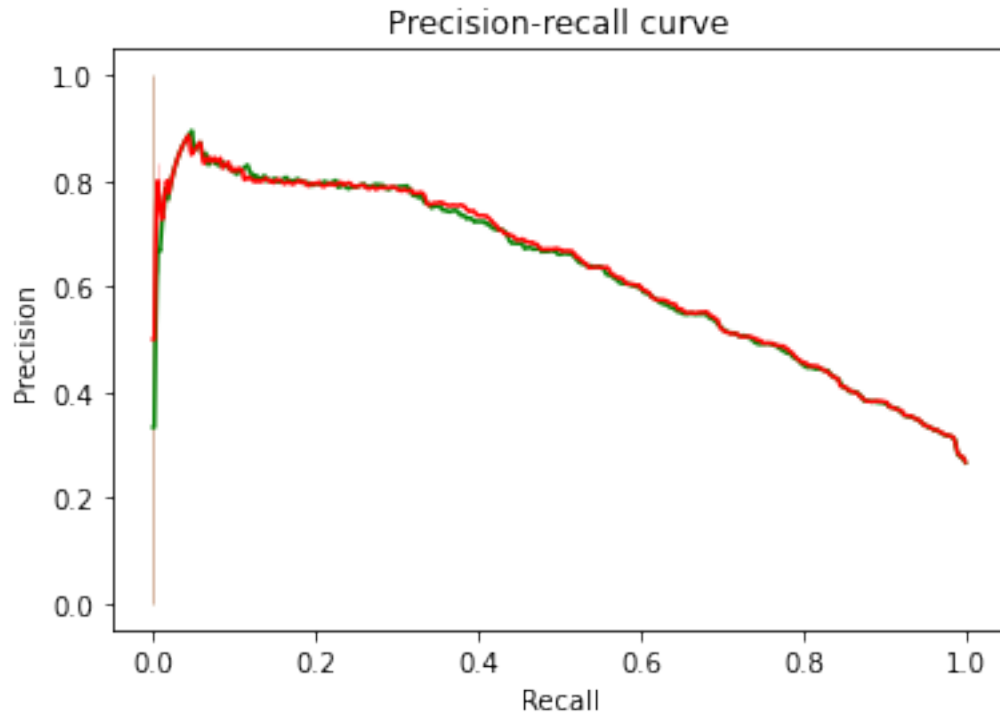
fig = plt.figure()
ax = fig.add_subplot(111)

sns.lineplot(x=recall_default, y=precision_default, color='g')
sns.lineplot(x=recall_best, y=precision_best, color='r')

plt.title("Precision-recall curve")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.show()

# end solution
```





## 1.8 Part 8. Plot ROC Curves and Compute the AUC for Both Models

You will next use scikit-learn's `roc_curve()` function to plot the receiver operating characteristic (ROC) curve and the `auc()` function to compute the area under the curve (AUC) for both models.

- An ROC curve plots the performance of a binary classifier for varying classification thresholds. It plots the fraction of true positives out of the positives vs. the fraction of false positives out of the negatives. For more information on how to use the `roc_curve()` function, consult the [scikit-learn documentation](#).
- The AUC measures the trade-off between the true positive rate and false positive rate. It provides a broad view of the performance of a classifier since it evaluates the performance for all the possible threshold values; it essentially provides a value that summarizes the the ROC curve. For more information on how to use the `auc()` function, consult the [scikit-learn documentation](#).

Let's first import the functions.

```
[17]: from sklearn.metrics import roc_curve
      from sklearn.metrics import auc
```

Task: Using the `roc_curve()` function, record the true positive and false positive rates for both models.

1. Call `roc_curve()` with arguments `y_test` and `proba_predictions_default`. The `roc_curve` function produces three outputs. Save the three items to the following variables, respectively: `fpr_default` (standing for 'false positive rate'), `tpr_default` (standing for 'true positive rate'), and `thresholds_default`.
2. Call `roc_curve()` with arguments `y_test` and `proba_predictions_best`. The `roc_curve` function produces three outputs. Save the three items to the following variables, respectively: `fpr_best` (standing for 'false positive rate'), `tpr_best` (standing for 'true positive rate'), and `thresholds_best`.

```
[18]: #fpr_default, tpr_default, thresholds_default = # YOUR CODE HERE
      #fpr_best, tpr_best, thresholds_best = # YOUR CODE HERE

      # solution:

      fpr_default, tpr_default, thresholds_default = roc_curve(y_test,
      ↪proba_predictions_default)
      fpr_best, tpr_best, thresholds_best = roc_curve(y_test, proba_predictions_best)
```

Task: Create two `seaborn` lineplots to visualize the ROC curve for both models.

The plot for the default hyperparameter should be green. The plot for the best hyperparameter should be red.

- In each plot, the `fpr` values should be on the  $x$ -axis.
- In each plot, the `tpr` values should be on the  $y$ -axis.
- In each plot, label the  $x$ -axis "False positive rate".
- In each plot, label the  $y$ -axis "True positive rate".
- Give each plot the title "Receiver operating characteristic (ROC) curve".
- Create a legend on each plot indicating that the plot represents either the default hyperparameter value or the best hyperparameter value.

Note: It may take a few minutes to produce each plot.

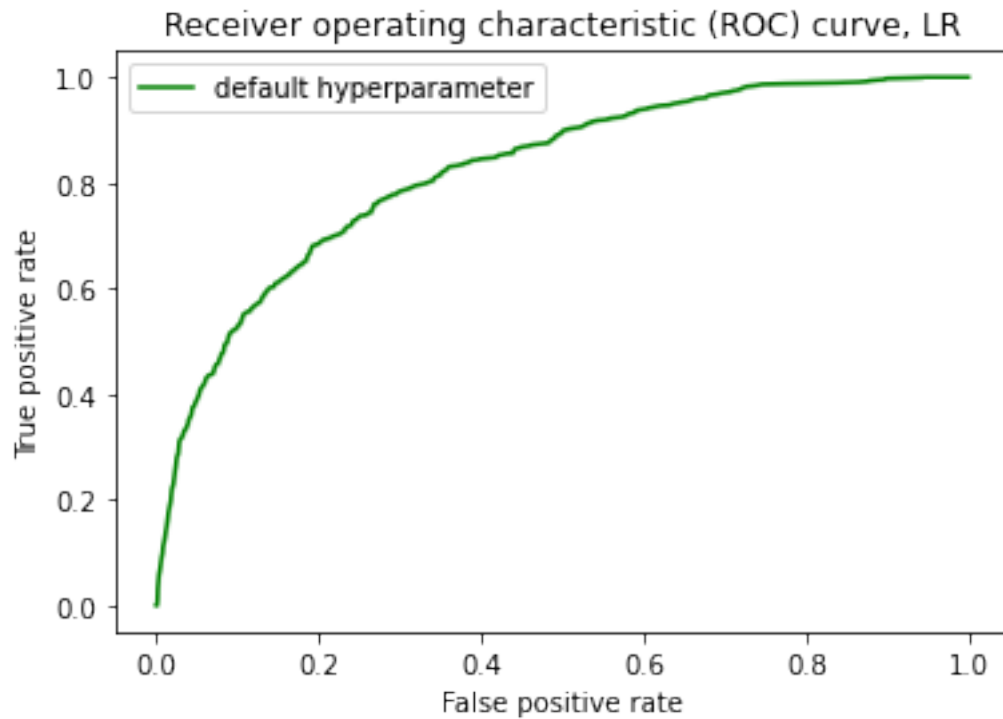
### Plot ROC Curve for Default Hyperparameter:

```
[19]: # YOUR CODE HERE

      ### Solution:
      fig = plt.figure()
      ax = fig.add_subplot(111)

      sns.lineplot(x=fpr_default, y=tpr_default, color='g')

      plt.title("Receiver operating characteristic (ROC) curve, LR")
      plt.xlabel("False positive rate")
      plt.ylabel("True positive rate")
      plt.legend(['default hyperparameter'])
      plt.show()
      ## end solution
```



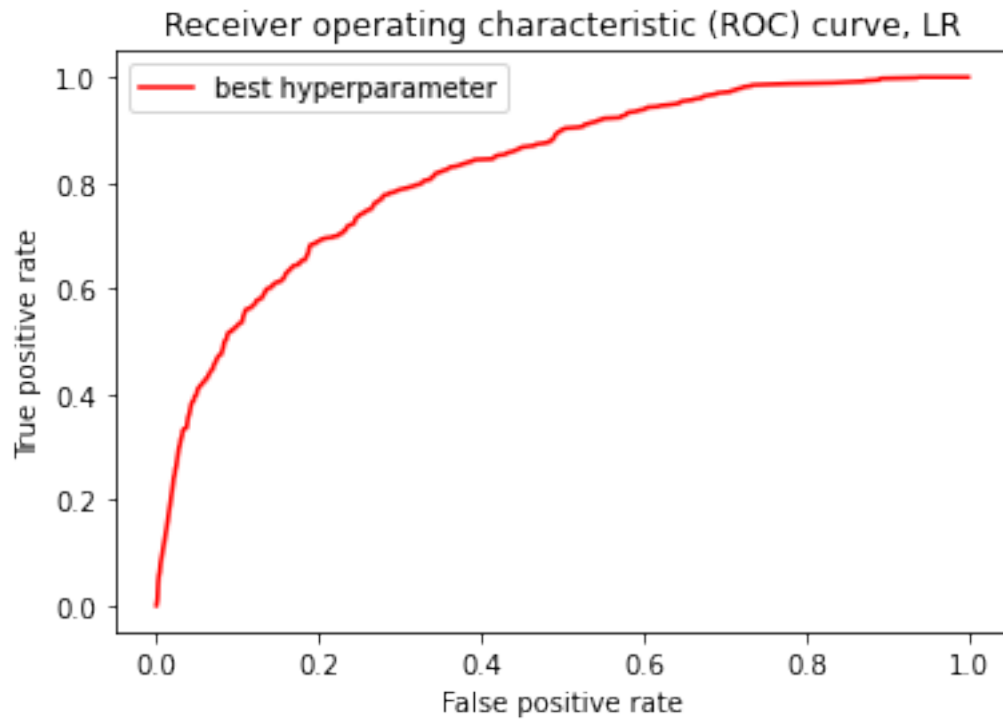
**Plot ROC Curve for Best Hyperparameter:**

[20]: *# YOUR CODE HERE*

```
### Solution:
fig = plt.figure()
ax = fig.add_subplot(111)

sns.lineplot(x=fpr_best, y=tpr_best, color='r')

plt.title("Receiver operating characteristic (ROC) curve, LR")
plt.xlabel("False positive rate")
plt.ylabel("True positive rate")
plt.legend(['best hyperparameter'])
plt.show()
# end solution
```



Task: Use the `auc()` function to compute the area under the receiver operating characteristic (ROC) curve for both models.

For each model, call the function with the `fpr` argument first and the `tpr` argument second.

Save the result of the `auc()` function for `model_default` to the variable `auc_default`. Save the result of the `auc()` function for `model_best` to the variable `auc_best`. Compare the results.

```
[21]: # YOUR CODE HERE

### Solution:
auc_default = auc(fpr_default, tpr_default)
auc_best = auc(fpr_best, tpr_best)
# end solution

print(auc_default)
print(auc_best)
```

0.8227761701899632

0.8242484526967285

## 1.9 Deep Dive: Feature Selection Using SelectKBest

In the code cell below, you will see how to use scikit-learn's `SelectKBest` class to obtain the best features in a given data set using a specified scoring function. For more information on how to use `SelectKBest`, consult the online [documentation](#).

We will extract the best 5 features from the Airbnb "listings" data set to create new training data, then fit our model with the optimal hyperparameter  $C$  to the data and compute the AUC. Walk through the code to see how it works and complete the steps where prompted. Analyze the results.

```
[22]: from sklearn.feature_selection import SelectKBest
      from sklearn.feature_selection import f_classif

      # Note that k=5 is specifying that we want the top 5 features
      selector = SelectKBest(f_classif, k=5)
      selector.fit(X, y)
      filter = selector.get_support()
      top_5_features = X.columns[filter]

      print("Best 5 features:")
      print(top_5_features)

      # Create new training and test data for features
      new_X_train = X_train[top_5_features]
      new_X_test = X_test[top_5_features]

      # Initialize a LogisticRegression model object with the best value of  $C$ 
      # → hyperparameter  $C$ 
      # The model object should be named 'model'
      # Note: Supply max_iter=1000 as an argument when creating the model object
      # YOUR CODE HERE

      # solution
      model = LogisticRegression(C = best_C, max_iter=1000)

      # Fit the model to the new training data
      # YOUR CODE HERE

      # solution
      model.fit(new_X_train, y_train)

      # Use the predict_proba() method to use your model to make predictions on the  $C$ 
      # → new test data
      # Save the values of the second column to a list called 'proba_predictions'
      # YOUR CODE HERE

      # solution
```

```
pp = model.predict_proba(new_X_test)
proba_predictions = []
for i in pp:
    proba_predictions.append(i[1])

# Compute the auc-roc
fpr, tpr, thresholds = roc_curve(y_test, proba_predictions)
auc_result = auc(fpr, tpr)
print(auc_result)
```

Best 5 features:

```
Index(['host_response_rate', 'number_of_reviews', 'number_of_reviews_ltm',
      'number_of_reviews_l30d', 'review_scores_cleanliness'],
      dtype='object')
0.7971528949977225
```

Task: Consider the results. Change the specified number of features and re-run your code. Does this change the AUC value? What number of features results in the best AUC value? Record your findings in the cell below.

solutions will vary based on the values students tested.

## 1.10 Part 9. Make Your Model Persistent

You will next practice what you learned in the "Making Your Model Persistent" activity, and use the `pickle` module to save `model_best`.

First we will import the `pickle` module.

```
[23]: import pickle
```

Task: Use `pickle` to save your model to a `pkl` file in the current working directory. Choose the name of the file.

```
[24]: # YOUR CODE HERE

# SOLUTION
pkl_model_filename = "LR_Model.pkl"

pickle.dump(model_best, open(pkl_model_filename, 'wb'))
```

Task: Test that your model is packaged and ready for future use by:

1. Loading your model back from the file
2. Using your model to make predictions on `X_test`.

```
[25]: # YOUR CODE HERE
```

```
# SOLUTION
```

```
persistent_model = pickle.load(open(pk1_model_filename, 'rb'))
prediction = persistent_model.predict(X_test)
print(prediction)
```

[False False False ... False True False]

Task: Download your `pkl` file and your `airbnbData_train` data set, and push these files to your GitHub repository. You can download these files by going to **File -> Open**. A new tab will open in your browser that will allow you to select your files and download them.